

Did you use a language model?

A : I did not as there was no need for it.

What did you do to verify that your code worked as expected, whether you used a language model or not?

Referred to the solution provided for better clarity.

Chapter 2: Mortality

```
In [1]: import pandas as pd # Import the pandas library for data manipulation and analysis
```

Get the data

```
In [2]: mortality_url = "https://data.cdc.gov/api/views/v6ab-adf5/rows.csv?accessType=DOWNLOAD"
mortality_data = pd.read_csv(mortality_url) # Define the URL for the mortality data
```

Save and restore the DataFrame

```
In [3]: mortality_data.to_pickle('mortality_data.pkl') # Save the loaded mortality data
```

```
In [4]: mortality_data = pd.read_pickle('mortality_data.pkl') # Load the mortality data
```

Examine and clean the data

Display the data

```
In [5]: mortality_data # Display the first & last few rows of the mortality data DataFrame
```

Out[5]:

	Year	Age Group	Death Rate
0	1900	1-4 Years	1983.8
1	1901	1-4 Years	1695.0
2	1902	1-4 Years	1655.7
3	1903	1-4 Years	1542.1
4	1904	1-4 Years	1591.5
...	...	...	...
471	2014	15-19 Years	45.5
472	2015	15-19 Years	48.3
473	2016	15-19 Years	51.2
474	2017	15-19 Years	51.5
475	2018	15-19 Years	49.2

476 rows × 3 columns

In [6]: `mortality_data.head()` *# Display the first five rows of the mortality data DataFrame*

Out[6]:

	Year	Age Group	Death Rate
0	1900	1-4 Years	1983.8
1	1901	1-4 Years	1695.0
2	1902	1-4 Years	1655.7
3	1903	1-4 Years	1542.1
4	1904	1-4 Years	1591.5

In [7]: `mortality_data.tail(3)` *# Display the last three rows of the mortality data DataFrame*

Out[7]:

	Year	Age Group	Death Rate
473	2016	15-19 Years	51.2
474	2017	15-19 Years	51.5
475	2018	15-19 Years	49.2

In [8]: `display(mortality_data)` *# Use the display function to show the mortality data DataFrame*

	Year	Age Group	Death Rate
0	1900	1-4 Years	1983.8
1	1901	1-4 Years	1695.0
2	1902	1-4 Years	1655.7
3	1903	1-4 Years	1542.1
4	1904	1-4 Years	1591.5
...	...	...	...
471	2014	15-19 Years	45.5
472	2015	15-19 Years	48.3
473	2016	15-19 Years	51.2
474	2017	15-19 Years	51.5
475	2018	15-19 Years	49.2

476 rows × 3 columns

```
In [9]: # Set display options to customize the output of the DataFrame display
with pd.option_context(
    'display.max_rows', 5,
    'display.max_columns', None):
    display(mortality_data)
```

	Year	Age Group	Death Rate
0	1900	1-4 Years	1983.8
1	1901	1-4 Years	1695.0
...	...	...	...
474	2017	15-19 Years	51.5
475	2018	15-19 Years	49.2

476 rows × 3 columns

Display the DataFrame attributes

```
In [10]: mortality_data.values # Access the values of the mortality data DataFrame as a N
```

```
Out[10]: array([[1900, '1-4 Years', 1983.8],
                [1901, '1-4 Years', 1695.0],
                [1902, '1-4 Years', 1655.7],
                ...,
                [2016, '15-19 Years', 51.2],
                [2017, '15-19 Years', 51.5],
                [2018, '15-19 Years', 49.2]], dtype=object)
```

```
In [11]: # Print the index, column names, and size of the mortality data DataFrame
print("Index: ", mortality_data.index)
```

```
print("Columns:", mortality_data.columns)
print("Size: ", mortality_data.size)
print("Shape: ", mortality_data.shape)
```

```
Index: RangeIndex(start=0, stop=476, step=1)
Columns: Index(['Year', 'Age Group', 'Death Rate'], dtype='object')
Size: 1428
Shape: (476, 3)
```

Use the columns attribute to change the column names

```
In [14]: # Remove spaces from column names in the mortality data DataFrame
mortality_data.columns = mortality_data.columns.str.replace(" ", "")
```

```
In [15]: # Print the modified column names of the mortality data DataFrame
print(mortality_data.columns)
```

```
Index(['Year', 'AgeGroup', 'DeathRate'], dtype='object')
```

```
In [16]: mortality_data.head() # Display the first five rows of the modified mortality da
```

```
Out[16]:
```

	Year	AgeGroup	DeathRate
0	1900	1-4 Years	1983.8
1	1901	1-4 Years	1695.0
2	1902	1-4 Years	1655.7
3	1903	1-4 Years	1542.1
4	1904	1-4 Years	1591.5

Use the info(), nunique(), and describe() methods

```
In [17]: mortality_data.info() # Display a concise summary of the mortality data DataFram
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 476 entries, 0 to 475
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Year        476 non-null    int64
1   AgeGroup    476 non-null    object
2   DeathRate   476 non-null    float64
dtypes: float64(1), int64(1), object(1)
memory usage: 11.3+ KB
```

```
In [18]: mortality_data.info(memory_usage='deep') # Display a detailed summary of the Dat
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 476 entries, 0 to 475
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Year        476 non-null    int64
1   AgeGroup    476 non-null    object
2   DeathRate   476 non-null    float64
dtypes: float64(1), int64(1), object(1)
memory usage: 38.7 KB

```

```
In [19]: mortality_data.nunique() # Count the number of unique values in each column of t
```

```

Out[19]: Year        119
AgeGroup      4
DeathRate    430
dtype: int64

```

```
In [20]: mortality_data.describe() # Generates descriptive statistics for numerical colum
```

```

Out[20]:

```

	Year	DeathRate
count	476.000000	476.000000
mean	1959.000000	192.924160
std	34.387268	293.224216
min	1900.000000	11.400000
25%	1929.000000	40.575000
50%	1959.000000	89.500000
75%	1989.000000	222.575000
max	2018.000000	1983.800000

```
In [21]: mortality_data.describe().T # Transpose the descriptive statistics summary for b
```

```

Out[21]:

```

	count	mean	std	min	25%	50%	75%	max
Year	476.0	1959.000000	34.387268	1900.0	1929.000	1959.0	1989.000	2018.0
DeathRate	476.0	192.92416	293.224216	11.4	40.575	89.5	222.575	1983.8

Save and restore the cleaned DataFrame

```
In [22]: mortality_data.to_pickle('mortality_cleaned.pkl') # Save the cleaned mortality d
```

```

In [23]: mortality_data = pd.read_pickle('mortality_cleaned.pkl') # Load the cleaned mort
mortality_data.head()

```

```
Out[23]:
```

	Year	AgeGroup	DeathRate
0	1900	1-4 Years	1983.8
1	1901	1-4 Years	1695.0
2	1902	1-4 Years	1655.7
3	1903	1-4 Years	1542.1
4	1904	1-4 Years	1591.5

Access the data

How to access columns

```
In [24]: mortality_data.DeathRate.head(2) # Display the first two values from the 'DeathR
```

```
Out[24]: 0    1983.8
         1    1695.0
         Name: DeathRate, dtype: float64
```

```
In [25]: type(mortality_data.DeathRate) # Get the data type of the 'DeathRate' column
```

```
Out[25]: pandas.core.series.Series
```

```
In [26]: mortality_data['DeathRate'].head(2) # Display the first two values from the 'Dea
```

```
Out[26]: 0    1983.8
         1    1695.0
         Name: DeathRate, dtype: float64
```

```
In [30]: type(mortality_data['DeathRate']) # Get the data type of the 'DeathRate' column
```

```
Out[30]: pandas.core.series.Series
```

```
In [28]: mortality_data[['Year', 'DeathRate']].head(2) # Display the first two rows for th
```

```
Out[28]:
```

	Year	DeathRate
0	1900	1983.8
1	1901	1695.0

```
In [29]: type(mortality_data[['Year', 'DeathRate']]) # Get the data type of the DataFrame
```

```
Out[29]: pandas.core.frame.DataFrame
```

How to access rows

```
In [31]: mortality_data.query('Year==1900') # Filter rows where 'Year' is 1900 using quer
```

```
Out[31]:
```

	Year	AgeGroup	DeathRate
0	1900	1-4 Years	1983.8
119	1900	5-9 Years	466.1
238	1900	10-14 Years	298.3
357	1900	15-19 Years	484.8

```
In [32]: mortality_data.query('Year == 2000 and AgeGroup != "1-4 Years"') # Filter rows
```

```
Out[32]:
```

	Year	AgeGroup	DeathRate
219	2000	5-9 Years	15.8
338	2000	10-14 Years	20.3
457	2000	15-19 Years	67.1

```
In [33]: mortality_data.query('Year == 1900 or Year == 2000').head() # Filter rows for th
```

```
Out[33]:
```

	Year	AgeGroup	DeathRate
0	1900	1-4 Years	1983.8
100	2000	1-4 Years	32.4
119	1900	5-9 Years	466.1
219	2000	5-9 Years	15.8
238	1900	10-14 Years	298.3

```
In [ ]: # use backticks if a column name contains spaces
# mortality_data.query('Year == 2000 and `Age Group` != "1-4 Years"')
```

How to access a subset of rows and columns

```
In [34]: mortality_data.query('Year == 1900').DeathRate.head() # Filter rows for the year
```

```
Out[34]:
```

0	1983.8
119	466.1
238	298.3
357	484.8

Name: DeathRate, dtype: float64

```
In [35]: mortality_data.query('Year == 1900')['DeathRate'].head() # Alternate syntax to f
```

```
Out[35]:
```

0	1983.8
119	466.1
238	298.3
357	484.8

Name: DeathRate, dtype: float64

```
In [36]: mortality_data.query('Year == 1900')[['DeathRate']].head() # Filter 'Year' 1900
```

Out[36]:

DeathRate	
0	1983.8
119	466.1
238	298.3
357	484.8

```
In [37]: mortality_data.query('Year == 1900')[['AgeGroup', 'DeathRate']].head() # Filter
```

Out[37]:

	AgeGroup	DeathRate
0	1-4 Years	1983.8
119	5-9 Years	466.1
238	10-14 Years	298.3
357	15-19 Years	484.8

How to access rows with the loc[] accessor

```
In [38]: mortality_data.loc[[0,5,10]] # Select rows with index labels 0, 5, and 10 using
```

Out[38]:

	Year	AgeGroup	DeathRate
0	1900	1-4 Years	1983.8
5	1905	1-4 Years	1498.9
10	1910	1-4 Years	1397.3

```
In [39]: mortality_data.loc[4:6] # Select a range of rows from index 4 to 6 using .loc
```

Out[39]:

	Year	AgeGroup	DeathRate
4	1904	1-4 Years	1591.5
5	1905	1-4 Years	1498.9
6	1906	1-4 Years	1580.0

```
In [40]: mortality_data.loc[0:20:5] # Select every 5th row up to index 20 using .loc
```

Out[40]:

	Year	AgeGroup	DeathRate
0	1900	1-4 Years	1983.8
5	1905	1-4 Years	1498.9
10	1910	1-4 Years	1397.3
15	1915	1-4 Years	924.2
20	1920	1-4 Years	987.2


```
In [41]: mortality_data.loc[mortality_data.Year == 1917] # Select rows where 'Year' is 19
```

```
Out[41]:
```

	Year	AgeGroup	DeathRate
17	1917	1-4 Years	1066.0
136	1917	5-9 Years	290.7
255	1917	10-14 Years	218.9
374	1917	15-19 Years	380.3

How to access columns with the loc[] accessor

```
In [42]: mortality_data.loc[:, ['Year', 'AgeGroup']] # Select all rows and specific colum
```

```
Out[42]:
```

	Year	AgeGroup
0	1900	1-4 Years
1	1901	1-4 Years
2	1902	1-4 Years
3	1903	1-4 Years
4	1904	1-4 Years
...	...	...
471	2014	15-19 Years
472	2015	15-19 Years
473	2016	15-19 Years
474	2017	15-19 Years
475	2018	15-19 Years

476 rows × 2 columns

How to access a subset of rows and columns with the loc[] accessor

```
In [43]: mortality_data.loc[[0,5,10], ['AgeGroup', 'DeathRate']] # Select specific rows and
```

```
Out[43]:
```

	AgeGroup	DeathRate
0	1-4 Years	1983.8
5	1-4 Years	1498.9
10	1-4 Years	1397.3

```
In [44]: mortality_data.loc[4:6, 'AgeGroup': 'DeathRate'] # Select a slice of rows and colu
```

Out[44]:

	AgeGroup	DeathRate
--	----------	-----------

4	1-4 Years	1591.5
---	-----------	--------

5	1-4 Years	1498.9
---	-----------	--------

6	1-4 Years	1580.0
---	-----------	--------

How to access rows with the `iloc[]` accessor

```
In [45]: mortality_data.iloc[[0,5,10]] # Select rows with integer positions 0, 5, and 10
```

Out[45]:

	Year	AgeGroup	DeathRate
--	------	----------	-----------

0	1900	1-4 Years	1983.8
---	------	-----------	--------

5	1905	1-4 Years	1498.9
---	------	-----------	--------

10	1910	1-4 Years	1397.3
----	------	-----------	--------

```
In [46]: mortality_data.iloc[4:6] # Select rows with integer positions 4 and 5 using .iloc
```

Out[46]:

	Year	AgeGroup	DeathRate
--	------	----------	-----------

4	1904	1-4 Years	1591.5
---	------	-----------	--------

5	1905	1-4 Years	1498.9
---	------	-----------	--------

```
In [47]: mortality_data.iloc[0:20:5] # Select every 5th row up to position 20 using .iloc
```

Out[47]:

	Year	AgeGroup	DeathRate
--	------	----------	-----------

0	1900	1-4 Years	1983.8
---	------	-----------	--------

5	1905	1-4 Years	1498.9
---	------	-----------	--------

10	1910	1-4 Years	1397.3
----	------	-----------	--------

15	1915	1-4 Years	924.2
----	------	-----------	-------

How to access a subset of rows and columns with the `iloc[]` accessor

```
In [48]: mortality_data.iloc[[0,5,10],[1,2]] # Select specific rows and columns by integer positions
```

Out[48]:

	AgeGroup	DeathRate
--	----------	-----------

0	1-4 Years	1983.8
---	-----------	--------

5	1-4 Years	1498.9
---	-----------	--------

10	1-4 Years	1397.3
----	-----------	--------

```
In [49]: mortality_data.iloc[4:7,1:3] # Select a range of rows and columns by integer positions
```

Out[49]:

	AgeGroup	DeathRate
--	----------	-----------

4	1-4 Years	1591.5
---	-----------	--------

5	1-4 Years	1498.9
---	-----------	--------

6	1-4 Years	1580.0
---	-----------	--------

In [50]: `mortality_data.iloc[-10:]` # Select the last 10 rows of the DataFrame using .iloc

Out[50]:

	Year	AgeGroup	DeathRate
--	------	----------	-----------

466	2009	15-19 Years	51.9
-----	------	-------------	------

467	2010	15-19 Years	49.4
-----	------	-------------	------

468	2011	15-19 Years	48.9
-----	------	-------------	------

469	2012	15-19 Years	47.2
-----	------	-------------	------

470	2013	15-19 Years	44.8
-----	------	-------------	------

471	2014	15-19 Years	45.5
-----	------	-------------	------

472	2015	15-19 Years	48.3
-----	------	-------------	------

473	2016	15-19 Years	51.2
-----	------	-------------	------

474	2017	15-19 Years	51.5
-----	------	-------------	------

475	2018	15-19 Years	49.2
-----	------	-------------	------

Prepare the data

Sort the data

In [51]: `mortality_data.sort_values('DeathRate', ascending=False).head(3)` # Sort the Data

Out[51]:

	Year	AgeGroup	DeathRate
--	------	----------	-----------

0	1900	1-4 Years	1983.8
---	------	-----------	--------

1	1901	1-4 Years	1695.0
---	------	-----------	--------

2	1902	1-4 Years	1655.7
---	------	-----------	--------

In [52]: `mortality_data.sort_values(['Year', 'DeathRate']).head(3)` # Sort the DataFrame by

Out[52]:

	Year	AgeGroup	DeathRate
--	------	----------	-----------

238	1900	10-14 Years	298.3
-----	------	-------------	-------

119	1900	5-9 Years	466.1
-----	------	-----------	-------

357	1900	15-19 Years	484.8
-----	------	-------------	-------

```
In [53]: # Sort the DataFrame by 'Year' (ascending) and 'DeathRate' (descending), display
mortality_data.sort_values(['Year', 'DeathRate'],
                           ascending=[True, False]).head()
```

```
Out[53]:
```

	Year	AgeGroup	DeathRate
0	1900	1-4 Years	1983.8
357	1900	15-19 Years	484.8
119	1900	5-9 Years	466.1
238	1900	10-14 Years	298.3
1	1901	1-4 Years	1695.0

```
In [54]: # Sort the DataFrame by 'DeathRate' (descending) and 'Year' (ascending), display
mortality_data.sort_values(['DeathRate', 'Year'],
                           ascending=[False, True]).head()
```

```
Out[54]:
```

	Year	AgeGroup	DeathRate
0	1900	1-4 Years	1983.8
1	1901	1-4 Years	1695.0
2	1902	1-4 Years	1655.7
4	1904	1-4 Years	1591.5
6	1906	1-4 Years	1580.0

Apply statistical methods

```
In [55]: mortality_data.DeathRate.mean() # Calculate the mean of the 'DeathRate' column
```

```
Out[55]: 192.92415966386554
```

```
In [56]: mortality_data[['AgeGroup', 'DeathRate']].max() # Find the maximum values of 'Age
```

```
Out[56]: AgeGroup      5-9 Years
DeathRate      1983.8
dtype: object
```

```
In [57]: mortality_data.count() # Count the non-null values in each column of the DataFra
```

```
Out[57]: Year      476
AgeGroup    476
DeathRate    476
dtype: int64
```

```
In [58]: mortality_data.quantile([.1, .9]) # Count the non-null values in each column of t
```

TypeError

Traceback (most recent call last)

Cell In[58], line 1

```
----> 1 mortality_data.quantile([.1,.9])
```

File ~\ANACONDA\Lib\site-packages\pandas\core\frame.py:11831, in DataFrame.quantile(self, q, axis, numeric_only, interpolation, method)

```
11827     raise ValueError(
11828         f"Invalid method: {method}. Method must be in {valid_method}."
11829     )
11830 if method == "single":
> 11831     res = data._mgr.quantile(qs=q, interpolation=interpolation)
11832 elif method == "table":
11833     valid_interpolation = {"nearest", "lower", "higher"}
```

File ~\ANACONDA\Lib\site-packages\pandas\core\internals\managers.py:1507, in BlockManager.quantile(self, qs, interpolation)

```
1504 new_axes = list(self.axes)
1505 new_axes[1] = Index(qs, dtype=np.float64)
-> 1507 blocks = [
1508     blk.quantile(qs=qs, interpolation=interpolation) for blk in self.blocks
1509 ]
1511 return type(self)(blocks, new_axes)
```

File ~\ANACONDA\Lib\site-packages\pandas\core\internals\managers.py:1508, in <listcomp>(.0)

```
1504 new_axes = list(self.axes)
1505 new_axes[1] = Index(qs, dtype=np.float64)
1507 blocks = [
-> 1508     blk.quantile(qs=qs, interpolation=interpolation) for blk in self.blocks
1509 ]
1511 return type(self)(blocks, new_axes)
```

File ~\ANACONDA\Lib\site-packages\pandas\core\internals\blocks.py:1587, in Block.quantile(self, qs, interpolation)

```
1584 assert self.ndim == 2
1585 assert is_list_like(qs) # caller is responsible for this
-> 1587 result = quantile_compat(self.values, np.asarray(qs._values), interpolation)
1588 # ensure_block_shape needed for cases where we start with EA and result
1589 # is ndarray, e.g. IntegerArray, SparseArray
1590 result = ensure_block_shape(result, ndim=2)
```

File ~\ANACONDA\Lib\site-packages\pandas\core\array_algos\quantile.py:39, in quantile_compat(values, qs, interpolation)

```
37 fill_value = na_value_for_dtype(values.dtype, compat=False)
38 mask = isna(values)
---> 39 return quantile_with_mask(values, mask, fill_value, qs, interpolation)
40 else:
41     return values._quantile(qs, interpolation)
```

File ~\ANACONDA\Lib\site-packages\pandas\core\array_algos\quantile.py:97, in quantile_with_mask(values, mask, fill_value, qs, interpolation)

```
95 result = np.repeat(fill_value, len(values)).reshape(len(values), len(qs))
96 else:
---> 97 result = _nanpercentile(
98     values,
```

```

99         qs * 100.0,
100         na_value=fill_value,
101         mask=mask,
102         interpolation=interpolation,
103     )
105     result = np.array(result, copy=False)
106     result = result.T

```

File ~\ANACONDA\Lib\site-packages\pandas\core\array_algos\quantile.py:218, in _nanpercentile(values, qs, na_value, mask, interpolation)

```

216     return result
217 else:
--> 218     return np.percentile(
219         values,
220         qs,
221         axis=1,
222         # error: No overload variant of "percentile" matches argument typ
es
223         # "ndarray[Any, Any]", "ndarray[Any, dtype[floating[_64Bit]]]",
224         # "int", "Dict[str, str]" [call-overload]
225         method=interpolation, # type: ignore[call-overload]
226     )

```

File <__array_function__ internals>:200, in percentile(*args, **kwargs)

File ~\ANACONDA\Lib\site-packages\numpy\lib\function_base.py:4205, in percentile(a, q, axis, out, overwrite_input, method, keepdims, interpolation)

```

4203 if not _quantile_is_valid(q):
4204     raise ValueError("Percentiles must be in the range [0, 100]")
-> 4205 return _quantile_unchecked(
4206     a, q, axis, out, overwrite_input, method, keepdims)

```

File ~\ANACONDA\Lib\site-packages\numpy\lib\function_base.py:4473, in _quantile_unchecked(a, q, axis, out, overwrite_input, method, keepdims)

```

4465 def _quantile_unchecked(a,
4466                         q,
4467                         axis=None,
4468                         (...),
4469                         method="linear",
4470                         keepdims=False):
4471     """Assumes that q is in [0, 1], and is an ndarray"""
-> 4473     return _ureduce(a,
4474                     func=_quantile_ureduce_func,
4475                     q=q,
4476                     keepdims=keepdims,
4477                     axis=axis,
4478                     out=out,
4479                     overwrite_input=overwrite_input,
4480                     method=method)

```

File ~\ANACONDA\Lib\site-packages\numpy\lib\function_base.py:3752, in _ureduce(a, func, keepdims, **kwargs)

```

3749         index_out = (0, ) * nd
3750         kwargs['out'] = out[(Ellipsis, ) + index_out]
-> 3752 r = func(a, **kwargs)
3754 if out is not None:
3755     return out

```

File ~\ANACONDA\Lib\site-packages\numpy\lib\function_base.py:4639, in _quantile_unchecked(a, q, axis, out, overwrite_input, method)

```

4637     else:
4638         arr = a.copy()
-> 4639 result = _quantile(arr,
4640                       quantiles=q,
4641                       axis=axis,
4642                       method=method,
4643                       out=out)
4644 return result

File ~\ANACONDA\Lib\site-packages\numpy\lib\function_base.py:4756, in _quantile(a
rr, quantiles, axis, method, out)
4754     result_shape = virtual_indexes.shape + (1,) * (arr.ndim - 1)
4755     gamma = gamma.reshape(result_shape)
-> 4756     result = _lerp(previous,
4757                     next,
4758                     gamma,
4759                     out=out)
4760 if np.any(slices_having_nans):
4761     if result.ndim == 0 and out is None:
4762         # can't write to a scalar

File ~\ANACONDA\Lib\site-packages\numpy\lib\function_base.py:4573, in _lerp(a, b,
t, out)
4559 def _lerp(a, b, t, out=None):
4560     """
4561     Compute the linear interpolation weighted by gamma on each point of
4562     two same shape array.
4563     (...)
4571     Output array.
4572     """
-> 4573     diff_b_a = subtract(b, a)
4574     # asanyarray is a stop-gap until gh-13105
4575     lerp_interpolation = asanyarray(add(a, diff_b_a * t, out=out))

TypeError: unsupported operand type(s) for -: 'str' and 'str'

```

```
In [59]: mortality_data['DeathRate'].cumsum() # Compute the cumulative sum of the 'DeathR
```

```

Out[59]: 0      1983.8
        1      3678.8
        2      5334.5
        3      6876.6
        4      8468.1
        ...
        471     91631.7
        472     91680.0
        473     91731.2
        474     91782.7
        475     91831.9
        Name: DeathRate, Length: 476, dtype: float64

```

Use Python for column arithmetic

```
In [60]: # Subtract the mean 'DeathRate' from each 'DeathRate' value to center the data a
mortality_data['MeanCentered'] = \
    mortality_data.DeathRate - mortality_data.DeathRate.mean()
```

```
In [61]: mortality_data # Display the DataFrame after adding the 'MeanCentered' column
```

Out[61]:

	Year	AgeGroup	DeathRate	MeanCentered
--	------	----------	-----------	--------------

0	1900	1-4 Years	1983.8	1790.87584
1	1901	1-4 Years	1695.0	1502.07584
2	1902	1-4 Years	1655.7	1462.77584
3	1903	1-4 Years	1542.1	1349.17584
4	1904	1-4 Years	1591.5	1398.57584
...	...	...	...	...
471	2014	15-19 Years	45.5	-147.42416
472	2015	15-19 Years	48.3	-144.62416
473	2016	15-19 Years	51.2	-141.72416
474	2017	15-19 Years	51.5	-141.42416
475	2018	15-19 Years	49.2	-143.72416

476 rows × 4 columns

```
In [62]: # Normalize 'DeathRate' by dividing each value by 100000
mortality_data['DeathRate'] = mortality_data.DeathRate / 100000
```

```
In [63]: mortality_data.head() # Display the first few rows of the DataFrame after norma
```

Out[63]:

	Year	AgeGroup	DeathRate	MeanCentered
--	------	----------	-----------	--------------

0	1900	1-4 Years	0.019838	1790.87584
1	1901	1-4 Years	0.016950	1502.07584
2	1902	1-4 Years	0.016557	1462.77584
3	1903	1-4 Years	0.015421	1349.17584
4	1904	1-4 Years	0.015915	1398.57584

Modify the string data in a column

```
In [64]: # Replace specific 'AgeGroup' values with a more standardized format using a dic
mortality_data.AgeGroup.replace(
    {'1-4 Years': '01-04 Years', '5-9 Years': '05-09 Years'},
    inplace = True)
```

```
In [65]: # Alternate method to replace 'AgeGroup' values using lists for 'to_replace' and
mortality_data.AgeGroup.replace(
    to_replace = ['1-4 Years', '5-9 Years'],
    value = ['01-04 Years', '05-09 Years'],
    inplace = True)
```

```
In [ ]: # mortality_data['AgeGroup'] = mortality_data.AgeGroup.str.replace('1-4 Years', '
# mortality_data['AgeGroup'] = mortality_data.AgeGroup.str.replace('5-9 Years', ')
```



```
In [66]: mortality_data # Display the DataFrame after standardizing 'AgeGroup' values
```

```
Out[66]:
```

	Year	AgeGroup	DeathRate	MeanCentered
0	1900	01-04 Years	0.019838	1790.87584
1	1901	01-04 Years	0.016950	1502.07584
2	1902	01-04 Years	0.016557	1462.77584
3	1903	01-04 Years	0.015421	1349.17584
4	1904	01-04 Years	0.015915	1398.57584
...	...	...	...	...
471	2014	15-19 Years	0.000455	-147.42416
472	2015	15-19 Years	0.000483	-144.62416
473	2016	15-19 Years	0.000512	-141.72416
474	2017	15-19 Years	0.000515	-141.42416
475	2018	15-19 Years	0.000492	-143.72416

476 rows × 4 columns

Save and restore the prepared DataFrame

```
In [67]: # Save the prepared mortality data to a pickle file for easy loading in the future
mortality_data.to_pickle('mortality_prepped.pkl')
```

```
In [68]: # Load the prepared mortality data from a pickle file and display the first few
mortality_data = pd.read_pickle('mortality_prepped.pkl')
mortality_data.head()
```

```
Out[68]:
```

	Year	AgeGroup	DeathRate	MeanCentered
0	1900	01-04 Years	0.019838	1790.87584
1	1901	01-04 Years	0.016950	1502.07584
2	1902	01-04 Years	0.016557	1462.77584
3	1903	01-04 Years	0.015421	1349.17584
4	1904	01-04 Years	0.015915	1398.57584

Shape the data

Set and use an index

```
In [69]: mortality_data.head(2) # Display the first two rows of the mortality data DataFrame
```

Out[69]:

	Year	AgeGroup	DeathRate	MeanCentered
0	1900	01-04 Years	0.019838	1790.87584
1	1901	01-04 Years	0.016950	1502.07584

In [70]: `# Set the 'Year' column as the DataFrame index and display the first two rows`
`mortality_data = mortality_data.set_index('Year')`
`mortality_data.head(2)`

Out[70]:

	AgeGroup	DeathRate	MeanCentered
Year			
1900	01-04 Years	0.019838	1790.87584
1901	01-04 Years	0.016950	1502.07584

In [71]: `mortality_data.reset_index(inplace=True) # Reset the DataFrame index, moving 'Ye`

In [72]: `# Set a multi-index using 'Year' and 'AgeGroup' columns with integrity check and`
`mortality_data = mortality_data.set_index(`
`['Year', 'AgeGroup'], verify_integrity=True)`
`mortality_data.head(2)`

Out[72]:

		DeathRate	MeanCentered
Year	AgeGroup		
1900	01-04 Years	0.019838	1790.87584
1901	01-04 Years	0.016950	1502.07584

In [73]: `mortality_data.reset_index(inplace=True) # Reset the DataFrame index, moving 'Y`

In [74]: `mortality_data.head(2) # Display the first two rows of the DataFrame after reset`

Out[74]:

	Year	AgeGroup	DeathRate	MeanCentered
0	1900	01-04 Years	0.019838	1790.87584
1	1901	01-04 Years	0.016950	1502.07584

Pivot the data

In [75]: `# Pivot the DataFrame to a wide format with 'Year' as index and 'AgeGroup' as co`
`mortality_wide = mortality_data.pivot(`
`index="Year", columns="AgeGroup")`
`mortality_wide.head(3)`

Out[75]:

AgeGroup	DeathRate							MeanC
	01-04 Years	05-09 Years	10-14 Years	15-19 Years	01-04 Years	05-09 Years	10-14 Years	
	Year							
1900	0.019838	0.004661	0.002983	0.004848	1790.87584	273.17584	105.37584	29
1901	0.016950	0.004276	0.002736	0.004544	1502.07584	234.67584	80.67584	26
1902	0.016557	0.004033	0.002525	0.004215	1462.77584	210.37584	59.57584	22

```
In [76]: # Pivot the DataFrame to wide format, selecting 'DeathRate' as values
mortality_wide = mortality_data.pivot(
    index="Year", columns="AgeGroup", values="DeathRate")
mortality_wide.head(3)
```

Out[76]:

AgeGroup	01-04 Years	05-09 Years	10-14 Years	15-19 Years
----------	-------------	-------------	-------------	-------------

Year				
1900	0.019838	0.004661	0.002983	0.004848
1901	0.016950	0.004276	0.002736	0.004544
1902	0.016557	0.004033	0.002525	0.004215

Melt the data

```
In [77]: mortality_wide.to_excel('mortality_wide.xlsx') # Export the wide-format DataFrame
```

```
In [78]: # Load the wide-format mortality data from an Excel file and display the first f
mortality_wide = pd.read_excel('mortality_wide.xlsx')
mortality_wide.head(4)
```

Out[78]:

	Year	01-04 Years	05-09 Years	10-14 Years	15-19 Years
--	------	-------------	-------------	-------------	-------------

0	1900	0.019838	0.004661	0.002983	0.004848
1	1901	0.016950	0.004276	0.002736	0.004544
2	1902	0.016557	0.004033	0.002525	0.004215
3	1903	0.015421	0.004147	0.002682	0.004341

```
In [79]: # Melt the wide-format DataFrame back to Long format with specified 'AgeGroup' c
mortality_long = mortality_wide.melt(
    id_vars = 'Year',
    value_vars=['01-04 Years', '05-09 Years'],
    var_name = 'AgeGroup',
    value_name='DeathRate')
mortality_long.head(4)
with pd.option_context('display.max_rows', 4):
    display(mortality_long)
```

	Year	AgeGroup	DeathRate
0	1900	01-04 Years	0.019838
1	1901	01-04 Years	0.016950
...	...	...	...
236	2017	05-09 Years	0.000116
237	2018	05-09 Years	0.000115

238 rows × 3 columns

Save and restore the wide DataFrame

```
In [80]: mortality_wide.to_pickle('mortality_wide.pkl') # Save the wide-format DataFrame
```

```
In [81]: # Load the wide-format DataFrame from a pickle file and display the first few rows
mortality_wide = pd.read_pickle('mortality_wide.pkl')
mortality_wide.head()
```

```
Out[81]:
```

	Year	01-04 Years	05-09 Years	10-14 Years	15-19 Years
0	1900	0.019838	0.004661	0.002983	0.004848
1	1901	0.016950	0.004276	0.002736	0.004544
2	1902	0.016557	0.004033	0.002525	0.004215
3	1903	0.015421	0.004147	0.002682	0.004341
4	1904	0.015915	0.004250	0.003052	0.004714

Analyze the data

Group the data

```
In [82]: mortality_data.head() # Display the first few rows of the mortality data DataFrame
```

```
Out[82]:
```

	Year	AgeGroup	DeathRate	MeanCentered
0	1900	01-04 Years	0.019838	1790.87584
1	1901	01-04 Years	0.016950	1502.07584
2	1902	01-04 Years	0.016557	1462.77584
3	1903	01-04 Years	0.015421	1349.17584
4	1904	01-04 Years	0.015915	1398.57584

```
In [83]: mortality_data.groupby('AgeGroup').mean() # Calculate the mean 'DeathRate' for each AgeGroup
```

Out[83]:

	Year	DeathRate	MeanCentered
--	------	-----------	--------------

AgeGroup			
01-04 Years	1959.0	0.003832	190.301891
05-09 Years	1959.0	0.001173	-75.598109
10-14 Years	1959.0	0.000938	-99.154412
15-19 Years	1959.0	0.001774	-15.549370

In [84]: `mortality_data.groupby('Year').median().head(4) # Calculate the median 'DeathRate'`

```

-----
TypeError                                Traceback (most recent call last)
File ~\ANACONDA\Lib\site-packages\pandas\core\groupby\groupby.py:1874, in GroupBy._agg_py_fallback(self, how, values, ndim, alt)
    1873 try:
-> 1874     res_values = self.grouper.agg_series(ser, alt, preserve_dtype=True)
    1875 except Exception as err:

File ~\ANACONDA\Lib\site-packages\pandas\core\groupby\ops.py:849, in BaseGrouper.agg_series(self, obj, func, preserve_dtype)
    847     preserve_dtype = True
--> 849 result = self._aggregate_series_pure_python(obj, func)
    851 if len(obj) == 0 and len(result) == 0 and isinstance(obj.dtype, ExtensionDtype):

File ~\ANACONDA\Lib\site-packages\pandas\core\groupby\ops.py:877, in BaseGrouper._aggregate_series_pure_python(self, obj, func)
    876 for i, group in enumerate(splitter):
--> 877     res = func(group)
    878     res = extract_result(res)

File ~\ANACONDA\Lib\site-packages\pandas\core\groupby\groupby.py:2460, in GroupBy.median.<locals>.<lambda>(x)
    2387 """
    2388 Compute median of groups, excluding missing values.
    2389 (...)
    2456 Freq: MS, dtype: float64
    2457 """
    2458 result = self._cython_agg_general(
    2459     "median",
-> 2460     alt=lambda x: Series(x).median(numeric_only=numeric_only),
    2461     numeric_only=numeric_only,
    2462 )
    2463 return result.__finalize__(self.obj, method="groupby")

File ~\ANACONDA\Lib\site-packages\pandas\core\series.py:6235, in Series.median(self, axis, skipna, numeric_only, **kwargs)
    6227 @doc(make_doc("median", ndim=1))
    6228 def median(
    6229     self,
    6230     ...,
    6231     **kwargs,
    6232 ):
-> 6235     return NDFrame.median(self, axis, skipna, numeric_only, **kwargs)

File ~\ANACONDA\Lib\site-packages\pandas\core\generic.py:12003, in NDFrame.median(self, axis, skipna, numeric_only, **kwargs)
    11996 def median(
    11997     self,
    11998     axis: Axis | None = 0,
    11999     ...,
    12000     **kwargs,
    12001 ) -> Series | float:
> 12003     return self._stat_function(
    12004         "median", nanops.nanmedian, axis, skipna, numeric_only, **kwargs
    12005     )

File ~\ANACONDA\Lib\site-packages\pandas\core\generic.py:11949, in NDFrame._stat_function(self, name, func, axis, skipna, numeric_only, **kwargs)

```

```

11947 validate_bool_kwarg(skipna, "skipna", none_allowed=False)
> 11949 return self._reduce(
11950     func, name=name, axis=axis, skipna=skipna, numeric_only=numeric_only
11951 )

```

File ~\ANACONDA\Lib\site-packages\pandas\core\series.py:6133, in Series._reduce(self, op, name, axis, skipna, numeric_only, filter_type, **kws)

```

6129     raise TypeError(
6130         f"Series.{name} does not allow {kwd_name}={numeric_only} "
6131         "with non-numeric dtypes."
6132     )
-> 6133 return op(delegate, skipna=skipna, **kws)

```

File ~\ANACONDA\Lib\site-packages\pandas\core\nanops.py:147, in bottleneck_switch.__call__.<locals>.f(values, axis, skipna, **kws)

```

146 else:
--> 147     result = alt(values, axis=axis, skipna=skipna, **kws)
149 return result

```

File ~\ANACONDA\Lib\site-packages\pandas\core\nanops.py:783, in nanmedian(values, axis, skipna, mask)

```

782     if inferred in ["string", "mixed"]:
--> 783         raise TypeError(f"Cannot convert {values} to numeric")
784 try:

```

TypeError: Cannot convert ['01-04 Years' '05-09 Years' '10-14 Years' '15-19 Years'] to numeric

The above exception was the direct cause of the following exception:

TypeError Traceback (most recent call last)
Cell In[84], line 1
----> 1 mortality_data.groupby('Year').median().head(4)

File ~\ANACONDA\Lib\site-packages\pandas\core\groupby\groupby.py:2458, in GroupBy.median(self, numeric_only)

```

2385 @final
2386 def median(self, numeric_only: bool = False):
2387     """
2388     Compute median of groups, excluding missing values.
2389     (...)
2456     Freq: MS, dtype: float64
2457     """
-> 2458     result = self._cython_agg_general(
2459         "median",
2460         alt=lamba x: Series(x).median(numeric_only=numeric_only),
2461         numeric_only=numeric_only,
2462     )
2463     return result._finalize__(self.obj, method="groupby")

```

File ~\ANACONDA\Lib\site-packages\pandas\core\groupby\groupby.py:1929, in GroupBy._cython_agg_general(self, how, alt, numeric_only, min_count, **kwargs)

```

1926     result = self._agg_py_fallback(how, values, ndim=data.ndim, alt=alt)
1927     return result
-> 1929 new_mgr = data.grouped_reduce(array_func)
1930 res = self._wrap_agged_manager(new_mgr)
1931 out = self._wrap_aggregated_output(res)

```

File ~\ANACONDA\Lib\site-packages\pandas\core\internals\managers.py:1428, in Block

```

kManager.grouped_reduce(self, func)
1424 if blk.is_object:
1425     # split on object-dtype blocks bc some columns may raise
1426     # while others do not.
1427     for sb in blk.split():
-> 1428         applied = sb.apply(func)
1429         result_blocks = extend_blocks(applied, result_blocks)
1430 else:

File ~\ANACONDA\Lib\site-packages\pandas\core\internals\blocks.py:366, in Block.apply(self, func, **kwargs)
360 @final
361 def apply(self, func, **kwargs) -> list[Block]:
362     """
363     apply the function to my values; return a block if we are not
364     one
365     """
--> 366     result = func(self.values, **kwargs)
367     result = maybe_coerce_values(result)
368     return self._split_op_result(result)

File ~\ANACONDA\Lib\site-packages\pandas\core\groupby\groupby.py:1926, in GroupBy._cython_agg_general.<locals>.array_func(values)
1923 else:
1924     return result
-> 1926 result = self._agg_py_fallback(how, values, ndim=data.ndim, alt=alt)
1927 return result

File ~\ANACONDA\Lib\site-packages\pandas\core\groupby\groupby.py:1878, in GroupBy._agg_py_fallback(self, how, values, ndim, alt)
1876 msg = f"agg function failed [how->{how},dtype->{ser.dtype}]"
1877 # preserve the kind of exception that raised
-> 1878 raise type(err)(msg) from err
1880 if ser.dtype == object:
1881     res_values = res_values.astype(object, copy=False)

TypeError: agg function failed [how->median,dtype->object]

```

In [85]: mortality_data.groupby(['Year', 'AgeGroup']).count().head() # Count the number of

Out[85]:

		DeathRate	MeanCentered
Year	AgeGroup		

Year	AgeGroup	DeathRate	MeanCentered
1900	01-04 Years	1	1
	05-09 Years	1	1
	10-14 Years	1	1
	15-19 Years	1	1
1901	01-04 Years	1	1

In [86]: mortality_data.groupby('AgeGroup')['DeathRate'].describe() # Generate descriptiv

Out[86]:

	count	mean	std	min	25%	50%	75%	max
AgeGroup								
01-04 Years	119.0	0.003832	0.005005	0.000240	0.000507	0.001091	0.005773	0.019838
05-09 Years	119.0	0.001173	0.001275	0.000114	0.000240	0.000484	0.001989	0.004661
10-14 Years	119.0	0.000938	0.000884	0.000139	0.000272	0.000446	0.001589	0.003751
15-19 Years	119.0	0.001774	0.001384	0.000448	0.000855	0.001069	0.002837	0.007774

Aggregate the data

In [87]: `mortality_data.groupby('AgeGroup').agg(['mean','median']) # Aggregate 'mean' an`

Out[87]:

	Year		DeathRate		MeanCentered	
	mean	median	mean	median	mean	median
AgeGroup						
01-04 Years	1959.0	1959.0	0.003832	0.001091	190.301891	-83.82416
05-09 Years	1959.0	1959.0	0.001173	0.000484	-75.598109	-144.52416
10-14 Years	1959.0	1959.0	0.000938	0.000446	-99.154412	-148.32416
15-19 Years	1959.0	1959.0	0.001774	0.001069	-15.549370	-86.02416

In [88]: `# Aggregate multiple statistics ('mean', 'median', 'std', 'nunique') for 'DeathR
mortality_data.groupby('AgeGroup')['DeathRate'] \`
`.agg(['mean','median','std','nunique'])`

Out[88]:

	mean	median	std	nunique
AgeGroup				
01-04 Years	0.003832	0.001091	0.005005	117
05-09 Years	0.001173	0.000484	0.001275	115
10-14 Years	0.000938	0.000446	0.000884	115
15-19 Years	0.001774	0.001069	0.001384	117

In [89]: `# Aggregate various statistics for 'DeathRate' within each 'Year'
mortality_data.groupby('Year')['DeathRate'] \`
`.agg(['mean','median','std','min','max','var','nunique'])`

Out[89]:

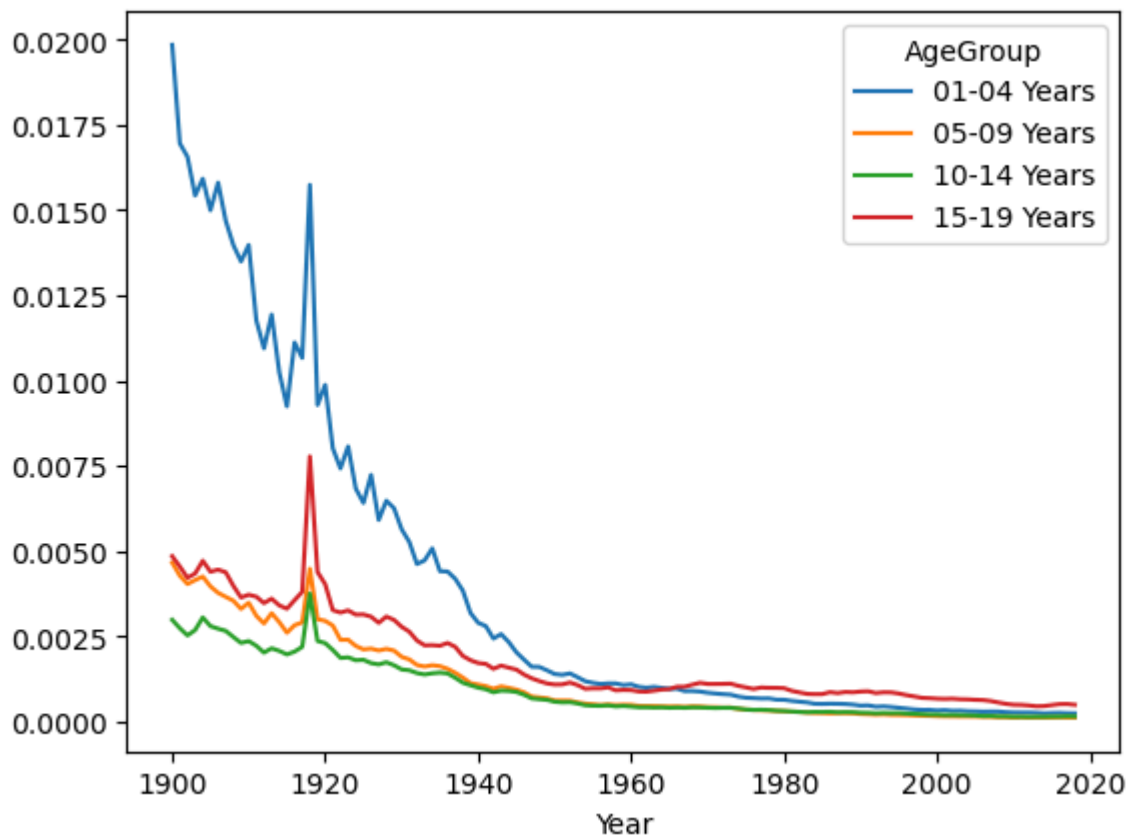
	mean	median	std	min	max	var	nunique
Year							
1900	0.008082	0.004755	0.007882	0.002983	0.019838	6.212178e-05	4
1901	0.007127	0.004410	0.006597	0.002736	0.016950	4.352410e-05	4
1902	0.006833	0.004124	0.006527	0.002525	0.016557	4.260299e-05	4
1903	0.006648	0.004244	0.005896	0.002682	0.015421	3.475731e-05	4
1904	0.006983	0.004482	0.005996	0.003052	0.015915	3.595034e-05	4
...	...	...	...	...	...	...	...
2014	0.000237	0.000190	0.000155	0.000115	0.000455	2.394167e-08	4
2015	0.000249	0.000197	0.000166	0.000117	0.000483	2.759625e-08	4
2016	0.000258	0.000200	0.000178	0.000122	0.000512	3.186025e-08	4
2017	0.000257	0.000199	0.000180	0.000116	0.000515	3.234825e-08	4
2018	0.000249	0.000195	0.000170	0.000115	0.000492	2.902867e-08	4

119 rows × 7 columns

Visualize the data

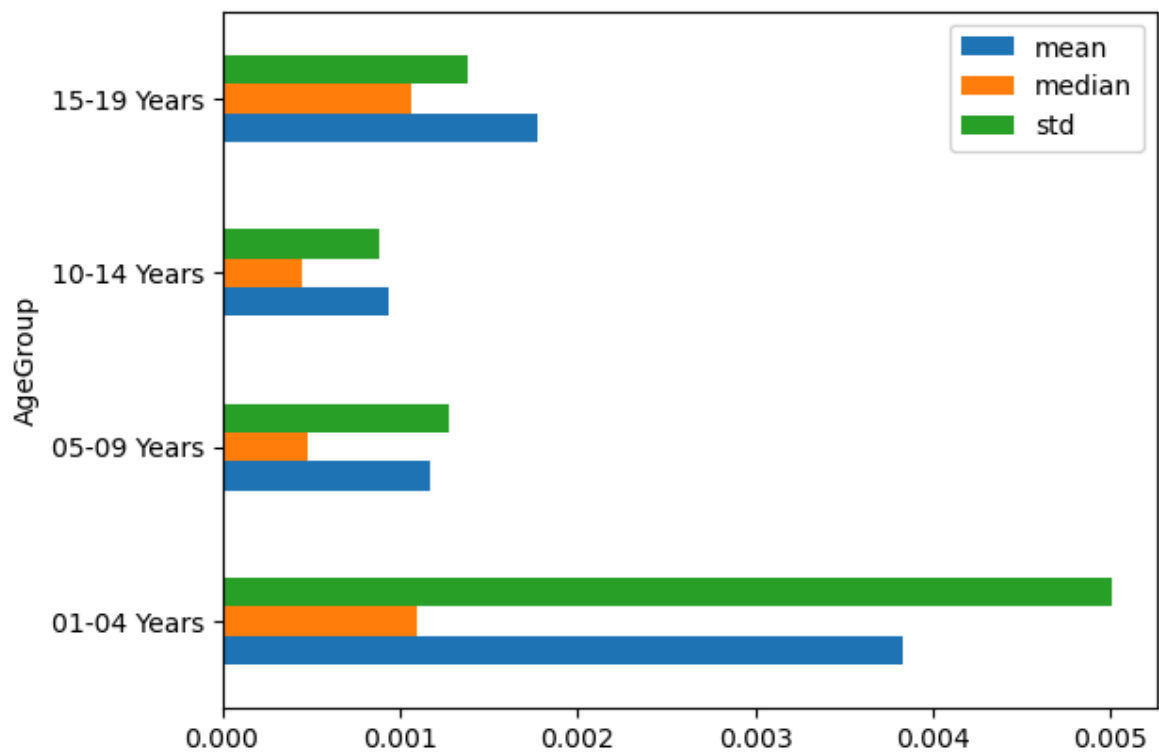
In [90]: `mortality_data.pivot(index='Year', columns='AgeGroup')['DeathRate'].plot()`

Out[90]: `<Axes: xlabel='Year'>`



```
In [91]: mortality_data.groupby('AgeGroup')['DeathRate'] \
        .agg(['mean', 'median', 'std']).plot.barh()
```

Out[91]: <Axes: ylabel='AgeGroup'>



In []: